

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Кафедра інформаційних систем та технологій

Курсова робота
З дисципліни «Технології розроблення програмного забезпечення»

Керівник

доц. Амонс О.А.

«Допущений до захисту»

(Особистий підпис керівника)

« » _____ 2025р.

Захищений з оцінкою

(оцінка)

Члени комісії:

(особистий підпис)

(особистий підпис)

Виконавець

ст. Ничик О. О.

залікова книжка № ІА – 3314

гр. ІА-33

(особистий підпис виконавця)

« 28 » листопада 2025р.

(розшифровка підпису)

(розшифровка підпису)

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КИЇВСЬКИЙ
ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

(назва навчального закладу)

Кафедра ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ

Дисципліна «Технології розроблення програмного забезпечення»

Курс 3 Група ІА-33 Семестр 5

ЗАВДАННЯ

на курсову роботу студента

Ничика Олександра Олеговича

(прізвище, ім'я, по батькові)

1. Тема роботи: Монітор активності системи.
2. Строк здачі студентом закінченої роботи: 28.11.2025.
3. Вихідні дані до роботи: система моніторингу активності користувача, реалізована мовою Java з використанням підходу SOA та шаблонів проєктування Abstract Factory, Iterator, Command, Visitor і Bridge. Розроблена система автоматично збирає статистику роботи комп'ютера: навантаження на процесор, обсяг використаної оперативної пам'яті, натискання клавіш, переміщення та кліки миші, зміну активних вікон і загальний час роботи системи. Усі дані фіксуються в локальній базі SQLite із прив'язкою до дат. Передбачено формування щоденних і періодичних звітів у різних форматах, що реалізовано за допомогою шаблону Visitor. Користувач може отримувати показники завантаження процесора, використання пам'яті по годинах, тривалість роботи програм, список відкритих вікон або загальний час активності за вибраний період. Крім автоматичного збору даних доступне примусове збереження статистики за запитом користувача.
4. Зміст розрахунково – пояснювальної записки (перелік питань, що підлягають розробці) включає: огляд існуючих рішень, загальний опис проєкту, вимоги до системи, опис сценаріїв використання, концептуальну модель системи, вибір бази даних, вибір мови програмування та середовища розробки, проєктування розгортання, діаграму розгортання системи, структуру бази даних, архітектуру системи, специфікацію системи, вибір і обґрунтування патернів реалізації, інструкцію користувача та додатки.

5.Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень):
діаграма класів, Use Case-діаграма, діаграма розгортання та структура бази даних.

6. Дата видачі завдання: 26.09.2025

КАЛЕНДАРНИЙ ПЛАН

№, п/п	Назва етапів виконання курсової роботи	Строк виконання етапів роботи	Підписи або примітки
1.	Отримання завдання на курсову роботу	26.09.2025	
2.	Підбір та вивчення літератури	30.09.2025	
3.	Огляд існуючих рішень та опис проекту	05.10.2025	
4.	Визначення вимог до системи та сценаріїв використання	10.10.2025	
5.	Концептуальна модель системи	15.10.2025	
6.	Вибір бази даних та мови програмування	25.10.2025	
7.	Проектування розгортання	30.10.2025	
8.	Структура БД та визначення архітектури системи	05.11.2025	
9.	Реалізація системи	20.11.2025	
10.	Представлення роботи до захисту	28.11.2025	

Студент _____
(підпис)

Олександр НИЧИК
(Ім'я ПРІЗВИЩЕ)

Керівник _____
(підпис)

Олександр АМОНС
(Ім'я ПРІЗВИЩЕ)

«26» вересня 2025 р.

ЗМІСТ

ВСТУП.....	7
1 ПРОЄКТУВАННЯ СИСТЕМИ.....	8
1.1 Огляд існуючих рішень.....	8
1.2 Загальний опис проєкту.....	9
1.3 Вимоги до застосунків системи, функціональні, нефункціональні вимоги.....	10
Для зручності вимоги представлено у вигляді діаграми:.....	10
1.4 Сценарії використання системи.....	12
1.5 Концептуальна модель системи.....	14
1.6 Вибір бази даних.....	20
1.7 Вибір мови програмування та середовища розробки.....	21
1.8 Проєктування розгортання системи.....	22
2 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ.....	24
2.1 Структура бази даних.....	24
2.2 Архітектура системи.....	24
2.2.1 Специфікація системи.....	24
2.2.2 Вибір та обґрунтування патернів реалізації.....	25
2.3 Інструкція користувача.....	36
ВИСНОВКИ.....	39
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	40
ДОДАТКИ.....	42

ВСТУП

У сучасних умовах розвитку комп'ютерних технологій важко уявити ефективну роботу користувача або підприємства без інструментів контролю стану комп'ютерної системи. Зростання кількості програм, підвищення навантаження на апаратні ресурси та збільшення рівня взаємодії з цифровими платформами робить моніторинг системної активності необхідним засобом для підтримання стабільності та продуктивності. Такі інструменти дають змогу своєчасно реагувати на збої, аналізувати поведінку системи та оптимізувати використання її ресурсів.

Метою цієї курсової роботи є розроблення програмного забезпечення «Монітор активності системи», що забезпечує збір та обробку ключових показників роботи комп'ютера. До основного функціоналу системи належать відстеження завантаженості процесора і оперативної пам'яті, моніторинг активних вікон і часу їх використання, фіксація діяльності користувача на клавіатурі та миші, а також облік загального часу роботи комп'ютера. Зібрана статистика надалі використовується для формування звітів, які дають можливість оцінювати стан системи та аналізувати використання ресурсів у різні проміжки часу.

У процесі роботи розглянуто підходи до проектування та реалізації системи моніторингу, визначено вимоги до програмного продукту, побудовано концептуальну модель, а також обґрунтовано вибір інструментів розробки, технологій та архітектурних рішень. Особливу увагу приділено застосуванню шаблонів проектування (Iterator, Factory, Command, Visitor), що забезпечують гнучкість і масштабованість системи.

Актуальність теми полягає у зростаючій потребі в інструментах, здатних наочно демонструвати стан комп'ютерних ресурсів і допомагати користувачам розуміти, як саме працює їх система. Отриманий програмний продукт може бути корисним для аналізу як персональних, так і робочих комп'ютерів, сприяючи підвищенню ефективності використання апаратного забезпечення та своєчасному виявленню можливих проблем.

1 ПРОЄКТУВАННЯ СИСТЕМИ

1.1 Огляд існуючих рішень

Сучасні комп'ютерні системи потребують постійного контролю стану апаратних і програмних ресурсів, тому на ринку представлено багато інструментів для моніторингу активності. Такі програми допомагають аналізувати навантаження на апаратні компоненти, слідкувати за стабільністю роботи системи та виявляти потенційні проблеми.

Серед базових рішень варто виділити стандартні утиліти, що входять до складу операційних систем. Наприклад, Windows Task Manager дозволяє швидко переглядати загальні показники – завантаження процесора, рівень використання оперативної пам'яті, кількість активних процесів. Однак цей інструмент орієнтований на оперативний аналіз і не зберігає історичні дані, що обмежує його застосування у випадках, коли необхідний довготривалий контроль.

Серед розширених рішень можна згадати Performance Monitor (PerfMon) - інструмент з широкими можливостями налаштування та збору системних лічильників. PerfMon підтримує трекінг багатьох метрик і формування звітів, однак складність інтерфейсу робить його менш зручним для звичайного користувача.

Open Hardware Monitor фокусується на моніторингу стану апаратних компонентів, але майже не працює з користувацькою активністю;

HWiNFO підтримує збирання історичних даних і глибокий аналіз, проте орієнтований переважно на технічних фахівців. Попри широкий вибір інструментів, більшість із них не поєднують одночасно збір активності користувача (клавіатура, миша, вікна), системних показників та гнучку роботу зі звітами. Також рідко зустрічається підтримка локального зберігання даних з прив'язкою до дат і можливістю формування структурованих аналітичних вибірок. Саме тому розробка власної системи моніторингу є актуальною, адже дозволяє адаптувати функціональність під конкретні потреби та інтегрувати необхідні патерни і архітектурні рішення.

1.2 Загальний опис проєкту

Проєкт «Монітор активності системи» спрямований на створення інструменту, що дозволяє відстежувати роботу комп'ютера в режимі реального часу та накопичувати історичні дані для аналізу. Система фіксує ключові параметри роботи, включаючи навантаження на процесор, обсяг використаної оперативної пам'яті, переміщення та кліки миші, натискання клавіш, зміну активних вікон і загальний час роботи комп'ютера протягом доби.

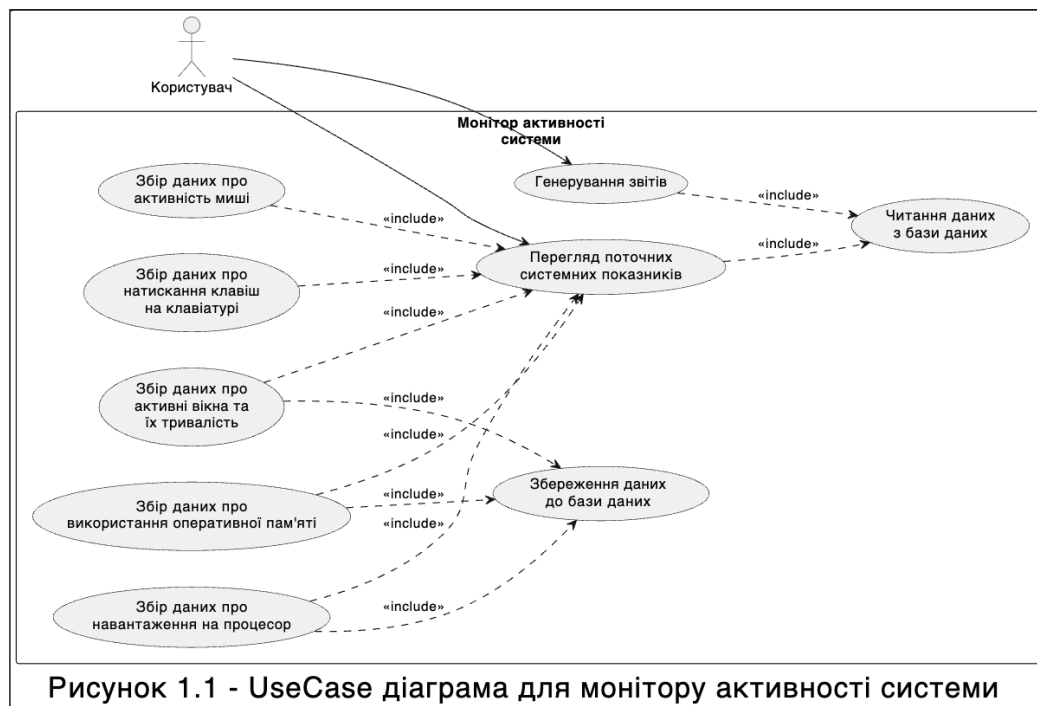
Зібрана статистика зберігається у локальній базі SQLite, що забезпечує можливість переглядати дані за будь-який день або за вибраний період. Доступний функціонал генерації щоденних і періодичних звітів у різних форматах, що реалізовано за допомогою патернів Command та Visitor. Це дозволяє користувачеві отримувати як текстові, так і JSON-звіти, залежно від потреби.

Програма також підтримує автоматичне визначення періодів бездіяльності, що дає змогу точніше оцінювати реальний час роботи та активності користувача. Окремо передбачена опція примусового збереження даних, яка дозволяє зафіксувати поточний стан системи незалежно від розкладу автоматичної обробки.

Проєкт орієнтований на користувачів, яким необхідно аналізувати роботу системи, контролювати продуктивність або вивчати патерни активності за певний період. Розроблена архітектура дозволяє легко розширювати функціонал, додавати нові монітори та формати звітів, що робить систему гнучким і масштабованим рішенням.

1.3 Вимоги до застосунків системи, функціональні, нефункціональні вимоги

Для зручності вимоги представлено у вигляді діаграми:



Актор "Користувач"

Це основний учасник системи, який взаємодіє з інтерфейсом монітора активності. Користувач може переглядати поточні системні показники, а також формувати звіти на основі накопичених даних.

Прецеденти (Use Cases)

- Перегляд поточних системних показників: Користувач отримує значення основних метрик у реальному часі, включаючи завантаження процесора, використання оперативної пам'яті, активні вікна та активність пристроїв вводу. Прецедент спирається на дані від відповідних модулів моніторингу.
- Генерування звітів: Користувач може сформувати звіт за обраний день або період. Для отримання інформації система звертається до прецеденту «Читання даних з бази даних».

- Збір даних про навантаження на процесор: Система фіксує поточне завантаження CPU. Дані використовуються як під час перегляду поточних значень, так і у звітності.
- Збір даних про використання оперативної пам'яті: Відстежується споживання RAM з періодичним записом результатів у базу даних для подальшого аналізу.
- Збір даних про активні вікна та їх тривалість: Фіксується активне вікно та час його використання. Дані зберігаються в БД і застосовуються у звітах.
- Збір даних про натискання клавіш на клавіатурі: Відстежується активність клавіатури, проте ці дані не записуються до бази даних.
- Збір даних про активність миші: Фіксується переміщення та натискання миші; дані використовуються лише для визначення активності користувача й не зберігаються в БД.
- Збереження даних до бази даних: Прецедент відповідає за запис значень, зібраних моніторами процесора, оперативної пам'яті, активних вікон та часу роботи системи. Запис може виконуватися автоматично або за запитом користувача.
- Читання даних з бази даних: Використовується під час формування звітів і перегляду історичних показників. Система завантажує дані відповідно до зазначених дат та передає їх модулю звітності.

Взаємозв'язки (include)

- Прецедент «Генерування звітів» включає (include) прецедент «Читання даних з бази даних», оскільки звіти формуються лише на основі збережених значень.
- Прецедент «Перегляд поточних системних показників» включає дані від усіх модулів моніторингу - процесора, пам'яті, активних вікон, клавіатури та миші.
- Монітори, що зберігають історію (CPU, RAM, Active Windows), включають прецедент «Збереження даних до бази даних».
- Монітори клавіатури та миші мають зв'язок лише з «Переглядом поточних системних показників», оскільки їхня активність не записується в базу.

1.4 Сценарії використання системи

Таблиця 1.1 - Сценарій використання «Перегляд поточних системних показників»

Назва	Перегляд поточних системних показників
ID	1
Опис	Користувач відкриває застосунок для перегляду поточних показників роботи системи, зокрема навантаження на процесор, споживання оперативної пам'яті, активні вікна, а також активність клавіатури та миші.
Актори	Користувач
Вигоди компанії	Миттєвий доступ до актуального стану системи підвищує ефективність використання комп'ютера та дозволяє вчасно реагувати на перевантаження або підозрілу активність.
Частота користування	Постійно
Тригери	Користувач запускає програму.
Передумови	Система моніторингу активності запущена й виконує збір даних у реальному часі.
Постумови	Користувач переглянув актуальні показники роботи системи.
Основний розвиток	Користувач отримує на екрані значення поточного навантаження на CPU, використання пам'яті, інформацію про активне вікно та індикатори активності клавіатури й миші.
Альтернативні розвитку	-
Виняткові ситуації	-

Таблиця 1.2 – Сценарій використання «Генерування звітів за певну дату»

Назва	Генерування звітів за певну дату
ID	2

Опис	Користувач запускає процес побудови звіту за конкретну дату. Система отримує дані з бази за вказаний день і формує звіт у вибраному форматі (текстовому або JSON).
Актори	Користувач
Вигоди компанії	Дозволяє детально аналізувати активність системи за конкретний день, виявляти пікові навантаження та оцінювати поведінку користувача протягом доби.
Частота користування	Постійно
Тригери	Користувач натискає кнопку генерації звіту та вибирає дату
Передумови	Система моніторингу активності працює та має збережені дані за вказану дату.
Постумови	Згенерований звіт відображається користувачу у вікні перегляду.
Основний розвиток	Користувач обирає дату, система зчитує пов'язані з нею записи з бази даних і формує потрібний звіт.
Альтернативні розвитку	Система показує повідомлення, що дані за вказану дату відсутні.
Виняткові ситуації	Базу даних пошкоджено або відсутній доступ до неї.

Таблиця 1.3 – Сценарій використання «Генерування звітів за певний період»

Назва	Генерування звітів за певний період
ID	3
Опис	Користувач ініціює формування звіту для аналізу використання системних ресурсів за вибраний часовий проміжок. Система отримує необхідні дані з бази та будує звіт у вибраному форматі.
Актори	Користувач
Вигоди компанії	Дає можливість аналізувати продуктивність роботи комп'ютера за певний інтервал часу, виявляти перевантаження та

	оптимізувати використання ресурсів.
Частота користування	Постійно
Тригери	Користувач натискає кнопку генерації звіту та вибирає діапазон дат.
Передумови	Система моніторингу активності запущена та має збережені дані за вказаний період.
Постумови	Згенерований звіт відображається користувачу у вікні перегляду.
Основний розвиток	Користувач обирає період і тип звіту, після чого система зчитує дані з бази та формує результат.
Альтернативні розвитку	Система показує повідомлення, що дані за вказаний період відсутні.
Виняткові ситуації	Базу даних пошкоджено або відсутній доступ до неї.

1.5 Концептуальна модель системи

Для опису концептуальної моделі було створено діаграму класів, яка відображає внутрішню структуру системи моніторингу активності та взаємозв'язки між її основними компонентами (див. Додаток А).

Діаграма показує ключові класи, що реалізують логіку збору системних показників, роботу з базою даних, формування звітів та управління життєвим циклом моніторингу. Рисунок 1.2 демонструє взаємодію основних структурних елементів системи, що реалізують патерни Factory, Controller та Iterator, а також їх зв'язки з моніторами та підсистемою репозиторіїв.

Основні компоненти загальної діаграми:

1. Центральним компонентом є клас **ActivityMonitorController**, який координує роботу всієї системи. Саме він відповідає за створення моніторів через фабрику, керування запуском та зупинкою моніторингу, взаємодію з графічним

інтерфейсом, передачу моніторів ітератору та формування звітів за допомогою **ReportService**. Контролер містить посилання на всі монітори, що працюють у системі, а також на файл бази даних і допоміжні об'єкти, що забезпечують коректну роботу застосунку.

2. Створення моніторів реалізоване через інтерфейс **MonitorFactory**, який визначає набір методів для побудови відповідних складових. Його конкретна реалізація, **ConcreteMacFactory**, формує монітори з урахуванням особливостей macOS та використовує **MonitorRepositoryFactory** для отримання доступу до репозиторіїв, з якими працюють ті монітори, що зберігають історичні дані. Такий підхід забезпечує ізоляцію логіки створення компонентів, спрощує підтримку та розширення системи і дозволяє легко адаптовувати її під інші платформи або додаткові модулі.
3. Для організації циклічного обходу моніторів застосовується **MonitorIterator**, який реалізує патерн **Iterator** та забезпечує послідовне опрацювання кожного об'єкта зі списку. Контролер використовує цей механізм у внутрішньому циклі моніторингу, що дає змогу виконувати оновлення даних у правильному порядку без прив'язки до конкретної структури зберігання.
4. Окрему роль відіграє **MonitorRepositoryFactory**, що створює та надає доступ до репозиторіїв, які працюють із базою даних. Це одного виду об'єкти, що забезпечують взаємодію з таблицями процесора, пам'яті, активних вікон, часу використання комп'ютера та дат моніторингу. Наявність такої фабрики дозволяє централізувати логіку створення репозиторіїв і зменшити дублювання коду.

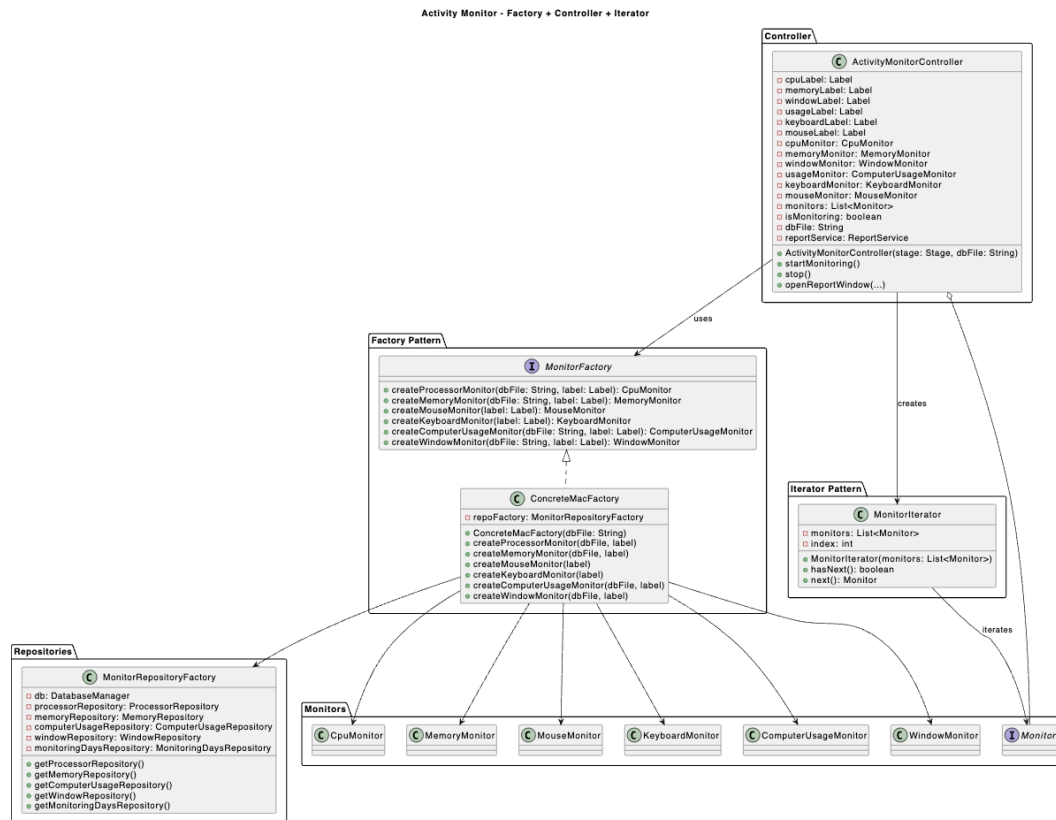


Рисунок 1.2 - Класи Factory, Controller та Iterator патернів

5. Діаграма моніторів демонструє структуру підсистеми збору даних у застосунку та взаємозв'язки між різними видами моніторів, спільними інтерфейсами та репозиторіями. Основою всієї ієрархії є інтерфейс **Monitor**, який визначає однакові для всіх моніторів методи **updateWidget()**, **saveData()** і **getActivityFlag()**. Через цей інтерфейс система може працювати з усіма видами моніторів уніфіковано, незалежно від того, які саме дані вони збирають чи зберігають. Окремі монітори, що вміють визначати активність користувача, реалізують додатковий інтерфейс **ActivityAwareMonitor**, який розширює базовий функціонал методом **checkActivity(isActive)**.
6. **KeyboardMonitor** та **MouseMonitor** функціонують як монітори, що відслідковують взаємодію користувача з пристроями введення, але не зберігають ці дані у базу. **KeyboardMonitor** фіксує натискання і відпускання клавіш, оновлюючи лише візуальне відображення активності. **MouseMonitor** збирає інформацію про кліки та рухи миші, використовуючи внутрішні лічильники, і так само не взаємодіє з репозиторіями. Обидва ці монітори

забезпечують визначення активності користувача в межах циклу моніторингу, але не створюють історичних записів.

7. **CpuMonitor**, **MemoryMonitor**, **WindowMonitor** та **ComputerUsageMonitor** належать до групи моніторів, що зберігають історичні дані, тому вони працюють у зв'язці з відповідними репозиторіями. **CpuMonitor** використовує **ProcessorRepository** для запису вимірів навантаження центрального процесора. **MemoryMonitor** оперує двома репозиторіями – **MemoryRepository**, у який записує значення використаної оперативної пам'яті, і **MonitoringDaysRepository**, що дозволяє прив'язувати записи до конкретної дати. **WindowMonitor** взаємодіє з **WindowRepository** для зберігання інформації про активні вікна та з **MonitoringDaysRepository** для реєстрації тривалості їх використання. **ComputerUsageMonitor**, окрім визначення активності користувача, фіксує час роботи системи через **ComputerUsageRepository**, записуючи кількість секунд активності у відповідний день.

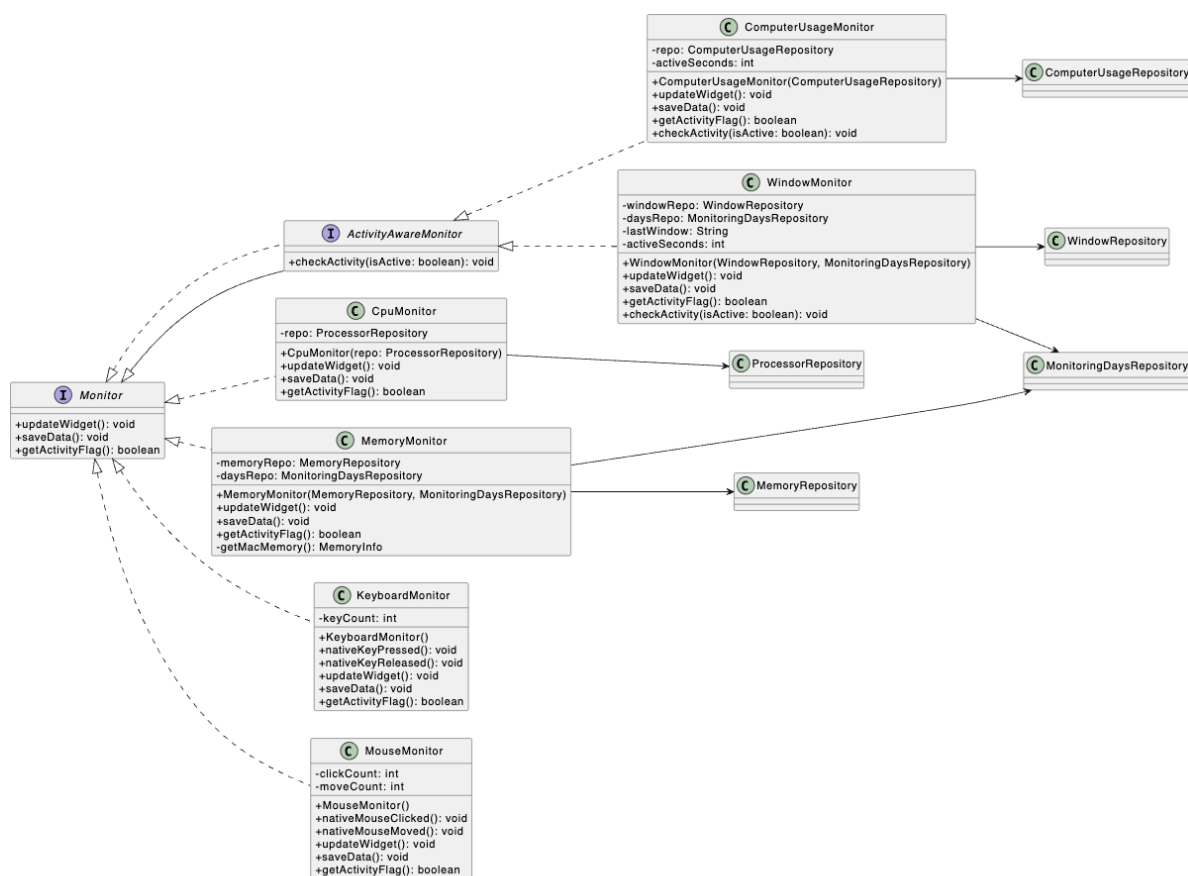


Рисунок 1.3 - Класи Monitors

8. Діаграма репозиторіїв відображає підсистему збереження та доступу до історичних даних, що збираються моніторами під час роботи додатка. Кожен репозиторій відповідає за роботу з окремим типом даних і взаємодіє з базою SQLite через спільний об'єкт **DatabaseManager**. Така структура забезпечує чітке розділення відповідальностей, спрощує підтримку і дозволяє незалежно розвивати окремі компоненти системи.
9. **MonitoringDaysRepository** забезпечує уніфіковане управління датами моніторингу. Він дозволяє отримати ідентифікатор дати, створити його при першому зверненні, а також отримати список ідентифікаторів для діапазону дат. Репозиторій використовується всіма моніторами, що записують дані у базу, і забезпечує коректне групування статистики.
10. **ProcessorRepository** реалізує функції збереження та зчитування даних про навантаження на процесор. Він приймає готові вимірювання від **CpuMonitor** і записує їх у таблицю з прив'язкою до дати та часу. Також репозиторій може надавати вибірку за конкретний день або обчислювати середні значення для заданого діапазону дат, що використовується при формуванні звітів.
11. **MemoryRepository** працює з історією використання оперативної пам'яті. Він приймає значення, сформовані **MemoryMonitor**, та зберігає їх разом із часовими мітками. Репозиторій підтримує отримання статистики за день і розрахунок середнього споживання пам'яті за кілька днів. Завдяки цьому модуль звітності може будувати аналіз тенденцій використання RAM.
12. **ComputerUsageRepository** відповідає за збереження часу активності системи протягом дня. Дані надходять від **ComputerUsageMonitor** і зберігаються як кількість секунд активної роботи. Репозиторій також надає можливість отримання сумарного часу активності за обраний день, що є основою для формування статистики "uptime".
13. **WindowRepository** зберігає інформацію про активні вікна та тривалість їх використання. Він дозволяє додавати нові записи про зміну вікна, визначати ідентифікатор вікна за його назвою, отримувати детальну історію за день, а

також отримувати список вікон, з якими працював користувач. Ця інформація є ключовою для звітів про роботу з програмами та розподіл часу між ними.

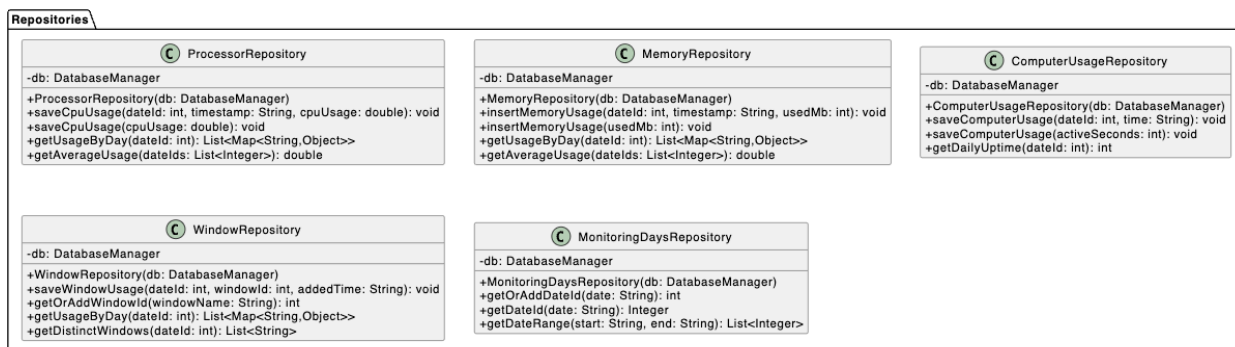


Рисунок 1.4 - Репозиторії для збереження даних моніторингу

14. Підсистема формування звітів об'єднує три ключові компоненти: логіку отримання статистики, команди для структурування запитів на генерацію звітів і відвідувачів, які визначають спосіб представлення отриманих даних. У центрі всієї підсистеми знаходиться клас **ReportService**. Він інкапсулює доступ до всіх репозиторіїв, що зберігають історію моніторингу, і надає методи отримання щоденних та періодичних звітів. Завдяки цьому **ReportService** виступає єдиною точкою взаємодії з даними про навантаження процесора, використання оперативної пам'яті, активність вікон і загальний час роботи системи.
15. Для того щоб процес генерації звітів був гнучким і розширюваним, використовується шаблон **Command**. Команди представляють окремі типи запитів: формування звіту за одну дату або за часовий період. Класи **GenerateDailyReportCommand** і **GeneratePeriodicReportCommand** містять параметри, необхідні для побудови звіту, і викликають відповідний метод у **ReportService**. Таким чином логіка запиту відокремлюється від логіки обробки даних, що дозволяє легко додавати нові типи звітів без зміни існуючої інтерфейсної частини.
16. Представлення звітів реалізовано через шаблон **Visitor**. Інтерфейс **ReportVisitor** визначає спосіб обробки та форматування наборів даних, які повертає **ReportService**. Конкретні реалізації, такі як **TextReportVisitor** або **JSONReportVisitor**, формують вихідний результат у потрібному форматі. Взаємодія команд із відвідувачами дозволяє зберігати універсальний формат

внутрішніх структур даних і при цьому надавати користувачу можливість отримувати звіти у різних формах. Це також забезпечує розширюваність, оскільки додавання нових форматів не впливає на логіку формування звітів чи структуру команд.

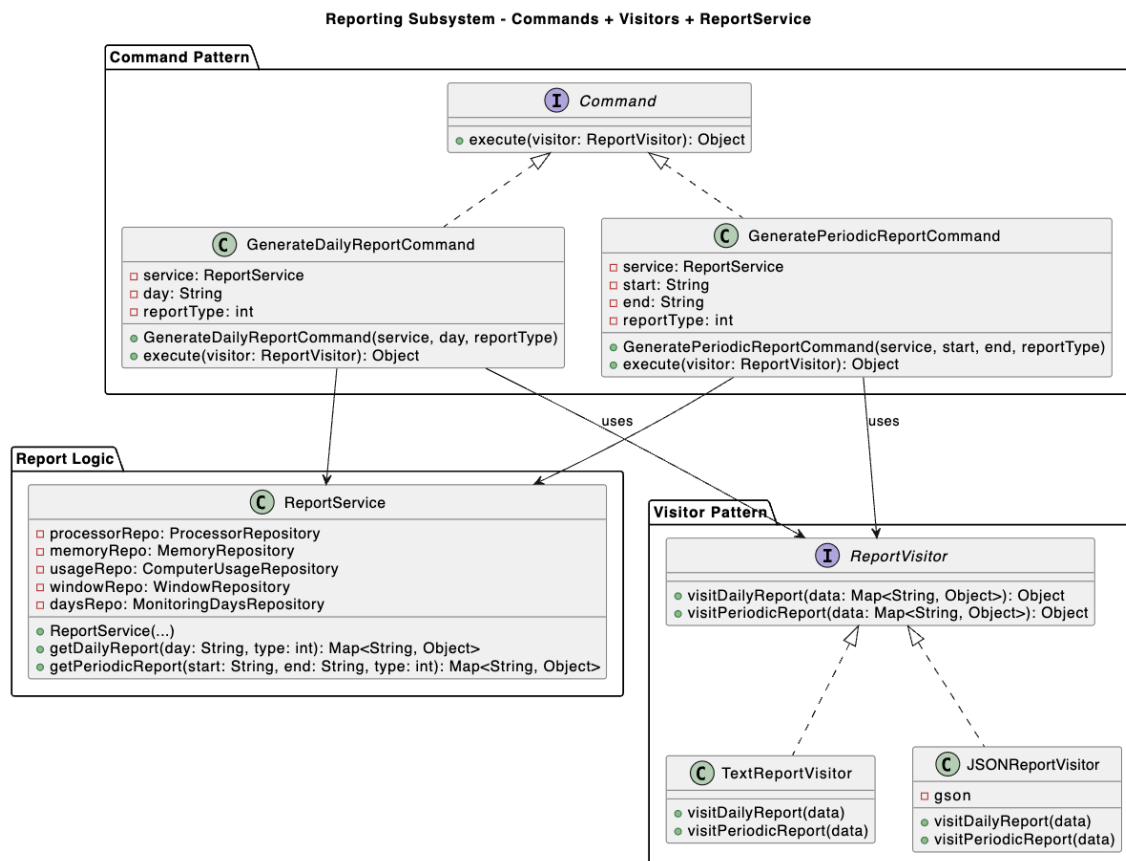


Рисунок 1.5 - Система звітів з використанням патернів Command та Visitor

1.6 Вибір бази даних

Для системи моніторингу активності було обрано базу даних SQLite, оскільки її характеристики повністю відповідають потребам застосунку. Це вбудована файлова СУБД, яка не вимагає ані розгортання серверної інфраструктури, ані додаткових сервісів. Використання однієї бази у вигляді звичайного файлу робить інтеграцію максимально простою, а саму систему - автономною та легко переносимою між пристроями.

Оскільки моніторинг генерує помірну кількість даних, пов'язаних із щоденними вимірюваннями основних ресурсів, навантаження на СУБД залишається невисоким. SQLite у таких сценаріях працює дуже швидко й стабільно,

забезпечуючи ефективне виконання операцій вставки, читання та вибірок без відчутних затримок. Формат зберігання у вигляді окремого .sqlite файла також спрощує резервування, перенесення та тестування системи, оскільки не вимагає додаткових інструментів і підходить для запуску на будь-якій платформі, де працює сама програма.

Підтримка стандартного SQL дозволяє без проблем формувати потрібні таблиці, виконувати складні запити й надалі розширювати схему, не змінюючи підхід до роботи з даними. Це важливо для побудови звітів, які ґрунтуються на агрегуванні та фільтрації показників за датами.

Обмеження SQLite, пов'язані з великим обсягом даних або необхідністю паралельного доступу багатьох клієнтів, у цьому проєкті не відіграють ролі, оскільки система працює локально й обслуговує лише один екземпляр програми. Тому вибір цієї СУБД є оптимальним і з точки зору продуктивності, і з точки зору простоти використання, і з точки зору підтримки проєкту в майбутньому.

1.7 Вибір мови програмування та середовища розробки

Для реалізації системи моніторингу активності було обрано мову програмування Java. Цей вибір зумовлений тим, що Java поєднує високу портативність, стабільність та розвинену екосистему бібліотек, що робить її придатною для створення кросплатформених настільних застосунків. Модель керування пам'яттю, підтримка багатопоточності та наявність численних інструментів для роботи з файлами, системними процесами та базами даних дають змогу ефективно реалізувати функції моніторингу.

Для побудови графічного інтерфейсу було використано JavaFX, оскільки цей фреймворк забезпечує сучасний вигляд застосунку, підтримує роботу з різними типами віджетів та надає зручні механізми оновлення інтерфейсу в реальному часі. JavaFX добре інтегрується з архітектурними підходами, використаними в системі, і дозволяє легко розмежувати логіку моніторингу та візуальну частину.

Як основне середовище розробки використано редактор Visual Studio Code, який надає оптимальний баланс між швидкістю та функціональністю. Завдяки підтримці розширень для Java він забезпечує зручну роботу з проектом, автоматичне форматування коду, підсвічування синтаксису, інтелектуальне автодоповнення та інтеграцію з Git.

1.8 Проєктування розгортання системи

Нижче наведена діаграма розгортання системи моніторингу активності:

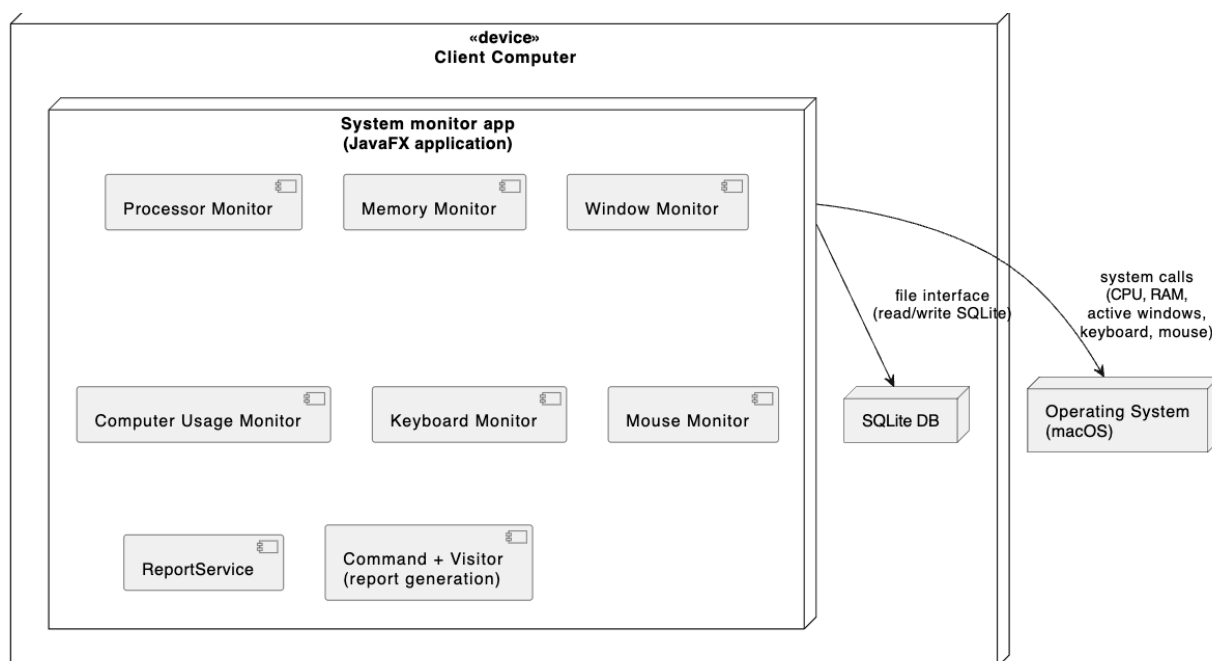


Рисунок 1.6 - Діаграма розгортання системи моніторингу активності

Діаграма розгортання демонструє фізичну архітектуру системи моніторингу активності та відображає, як її компоненти розміщуються на клієнтському комп'ютері та взаємодіють між собою під час роботи програми. Діаграма пояснює структуру виконання застосунку, способи доступу до системних ресурсів і механізм взаємодії з базою даних SQLite.

Усі елементи системи розміщуються на фізичному вузлі Client Computer, який представляє комп'ютер користувача, на якому запускається JavaFX-додаток моніторингу.

System Monitor App (JavaFX application) є головним програмним компонентом. Він містить набір моніторів, які відповідають за збір даних про використання процесора, оперативної пам'яті, активних вікон, час роботи комп'ютера, а також активність клавіатури й миші. У межах цього компонента також працює підсистема формування звітів, що реалізує патерни Command та Visitor і взаємодіє з локальною базою даних.

Operating System (macOS) забезпечує доступ до системних ресурсів, до яких звертаються окремі монітори. Саме через системні виклики програма отримує значення завантаження процесора та пам'яті, перелік активних вікон і поточну активність клавіатури й миші. Це дозволяє додатку формувати актуальні системні показники в реальному часі.

SQLite DB є локальною базою даних, яка використовується застосунком для зберігання історичних даних моніторингу. Взаємодія відбувається через файловий інтерфейс, оскільки SQLite працює як єдиний файл всередині клієнтського комп'ютера. Монітори, що збирають історичні значення, записують їх у цю базу, а підсистема звітів читає дані для формування щоденних і періодичних звітів.

Зв'язки між компонентами відображають два основні канали взаємодії: по-перше, звернення System Monitor App до операційної системи для отримання актуальних системних показників, і по-друге, взаємодію додатку з базою SQLite для зчитування й збереження моніторингових даних.

У разі необхідності систему можна масштабувати шляхом винесення бази даних у зовнішній сервер або додаванням віддаленої синхронізації. Однак для локального моніторингу така структура є оптимальною та не потребує складної інфраструктури.

2 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ

2.1 Структура бази даних

Нижче наведена майбутня база даних для створюваної системи:

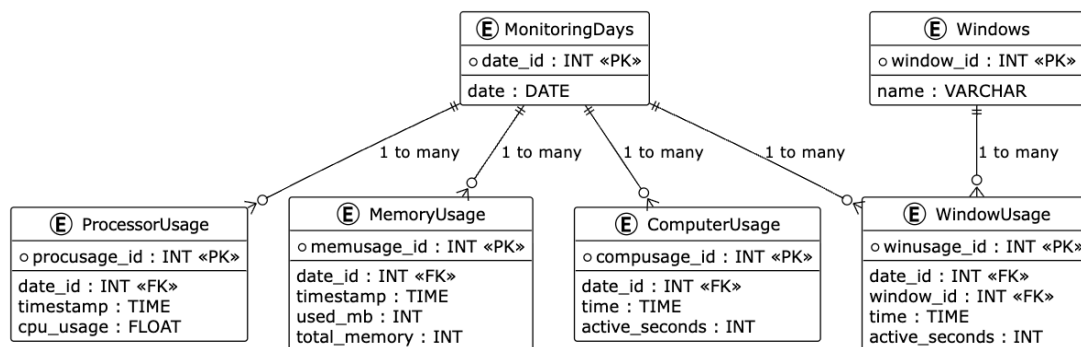


Рисунок 2.1 - Структура бази даних для збереження даних моніторингу

2.2 Архітектура системи

2.2.1 Специфікація системи

Специфікація системи	
Назва системи	Система моніторингу активності комп'ютера
Призначення	Програма призначена для автоматичного збору даних про активність комп'ютера, включаючи використання процесора, оперативної пам'яті, активні вікна та загальний час роботи системи. Зібрані дані зберігаються у локальній базі SQLite та можуть бути переглянуті або представлені у вигляді звітів за день або за період.
Цільова аудиторія	Користувачі, яким необхідно контролювати навантаження та стан комп'ютера у процесі роботи: для діагностики, оптимізації продуктивності або аналізу особистої активності.
Функціональні вимоги	1. Збір системних показників (CPU, RAM, активні вікна, робота системи). 2. Збереження даних у локальну базу SQLite. 3. Перегляд поточних значень у реальному часі. 4. Формування звітів за день та за певний період.
Нефункціональні вимоги	1. Продуктивність: - Збір даних має виконуватись у фоні без помітного впливу на роботу системи. - Оновлення вимірювань не повинно перевищувати інтервал у 1 секунду.

	2. Надійність: - Гарантована цілісність даних навіть у разі раптового завершення роботи програми. - Коректне відновлення після перезапуску. 3. Зручність користування: - Інтуїтивний графічний інтерфейс JavaFX. - Проста навігація між основними модулями (монітори, звіти).
--	---

2.2.2 Вибір та обґрунтування патернів реалізації

Шаблон “Iterator”

У системі моніторингу активності комп’ютера шаблон Iterator використано для послідовного обходу всіх моніторів системи (CPU, оперативна пам’ять, активні вікна, клавіатура, миша). Це дозволяє циклічно оновлювати показники в GUI без прив’язки до конкретних типів моніторів.

Iterator забезпечує такі переваги:

- колекція моніторів може легко змінюватися - додавання нового монітора не потребує змін логіки обходу;
- ActivityMonitorController не повинен знати внутрішню будову моніторів — лише те, що їх можна оновлювати;
- логіка обходу ізольована від логіки роботи моніторів, що спрощує підтримку системи.

У системі роль ітератора виконує клас MonitorIterator, який надає методи hasNext() та next() для послідовного доступу до моніторів.


```

public class MonitorIterator implements Iterator<Monitor> {

    private final List<Monitor> monitors;
    private int index = 0;

    public MonitorIterator(List<Monitor> monitors) {
        this.monitors = monitors;
    }

    @Override
    public boolean hasNext() {
        return index < monitors.size();
    }

    @Override
    public Monitor next() {
        return monitors.get(index++);
    }
}

```

Рисунок 2.2 - Клас MonitorIterator

Шаблон “Command”

У системі моніторингу активності комп’ютера шаблон Command використано для стандартизованого виконання операцій генерації звітів. Цей підхід дозволяє відокремити логіку формування запиту (команда) від логіки його виконання (сервіс звітів), що робить систему гнучкою і легко розширюваною.

У системі реалізовано дві конкретні команди:

- GenerateDailyReportCommand - створює звіт за одну дату;
- GeneratePeriodicReportCommand - створює звіт за період.

Обидві команди інкапсулюють в собі параметри запиту (дата, діапазон дат, тип звіту) та реалізують метод execute(ReportVisitor visitor), який передає виконання до сервісу звітів ReportService.

Для запуску команд використовується окремий клас ReportInvoker, який виступає у ролі Invoker за класичною структурою патерну. Invoker дозволяє встановити команду, яку необхідно виконати, та викликати її незалежно від типу

запиту. Це забезпечує можливість змінювати логіку виконання без зміни клієнтського коду.

Таким чином, застосування патерну Command у системі спрощує розширення функціоналу звітності: додавання нового типу звітів або нового формату представлення потребує лише створення нової команди, не зачіпаючи вже існуючу структуру.

```
public interface Command {  
    Object execute(ReportVisitor visitor);  
}
```

Рисунок 2.3 - Клас Command

```
public class GenerateDailyReportCommand implements Command {  
  
    private final ReportService service;  
    private final String day;  
    private final int reportType;  
  
    public GenerateDailyReportCommand(ReportService service,  
        String day, int reportType) {  
        this.service = service;  
        this.day = day;  
        this.reportType = reportType;  
    }  
  
    @Override  
    public Object execute(ReportVisitor visitor) {  
        var data = service.getDailyReport(day, reportType);  
        return visitor.visitDailyReport(data);  
    }  
}
```

Рисунок 2.4 - Клас GenerateDailyReportCommand

```
public class GeneratePeriodicReportCommand implements Command
{
    private final ReportService service;
    private final String start;
    private final String end;
    private final int reportType;

    public GeneratePeriodicReportCommand(ReportService
service, String start, String end, int reportType) {
        this.service = service;
        this.start = start;
        this.end = end;
        this.reportType = reportType;
    }

    @Override
    public Object execute(ReportVisitor visitor) {
        var data = service.getPeriodicReport(start, end,
reportType);
        return visitor.visitPeriodicReport(data);
    }
}
```

Рисунок 2.5 - Клас GeneratePeriodicReportCommand

```
public class ReportInvoker {

    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public Object executeCommand(ReportVisitor visitor) {
        if (command == null) return null;
        return command.execute(visitor);
    }
}
```

Рисунок 2.6 - Клас ReportInvoker

Шаблон “Abstract Factory”

Патерн Abstract Factory використано для створення наборів моніторів, специфічних для певної операційної системи (macOS).

Абстрактна фабрика визначає:

- які саме монітори існують у системі,
- як вони створюються,
- які параметри передаються у конструктори.

Це дозволяє розширити систему, додавши нову платформу (наприклад, WindowsFactory), не змінюючи клієнтський код.

В даній системі маємо:

- MonitorFactory - абстрактна фабрика;
- ConcreteMacFactory - конкретна реалізація під macOS;
- фабрика також отримує MonitorRepositoryFactory, що дозволяє будувати монітори разом із відповідними репозиторіями.

```
public interface MonitorFactory {  
  
    CpuMonitor createProcessorMonitor(String dbFile, Label  
guiLabel);  
  
    MemoryMonitor createMemoryMonitor(String dbFile, Label  
guiLabel);  
  
    MouseMonitor createMouseMonitor(Label guiLabel);  
  
    KeyboardMonitor createKeyboardMonitor(Label guiLabel);  
  
    ComputerUsageMonitor createComputerUsageMonitor(String  
dbFile, Label guiLabel);  
  
    WindowMonitor createWindowMonitor(String dbFile, Label  
guiLabel);  
}
```

Рисунок 2.7 - Клас MonitorFactory

```

public class ConcreteMacFactory implements MonitorFactory {

    @Override
    public MemoryMonitor createMemoryMonitor(String dbFile,
        Label guiLabel) {
        return new MemoryMonitor(
            guiLabel,
            repoFactory.getMemoryRepository(),
            repoFactory.getMonitoringDaysRepository()
        );
    }

    @Override
    public MouseMonitor createMouseMonitor(Label guiLabel) { ...

    @Override
    public KeyboardMonitor createKeyboardMonitor(Label
        guiLabel) { ...

    @Override
    public ComputerUsageMonitor createComputerUsageMonitor
        (String dbFile, Label guiLabel) {
        return new ComputerUsageMonitor(guiLabel, repoFactory.
            getComputerUsageRepository());
    }

    @Override
    public WindowMonitor createWindowMonitor(String dbFile,
        Label guiLabel) {
        return new WindowMonitor(
            guiLabel,
            repoFactory.getWindowRepository(),
            repoFactory.getMonitoringDaysRepository()
        );
    }
}

```

Рисунок 2.8 - Клас ConcreteMacFactory

```

public class MonitorRepositoryFactory {

    private final DatabaseManager db;

    private final ProcessorRepository processorRepository;
    private final MemoryRepository memoryRepository;
    private final ComputerUsageRepository
computerUsageRepository;
    private final WindowRepository windowRepository;
    private final MonitoringDaysRepository
monitoringDaysRepository;

    public MonitorRepositoryFactory(String dbFile) {
        this.db = new DatabaseManager(dbFile);

        this.processorRepository = new ProcessorRepository
(db);
        this.memoryRepository = new MemoryRepository(db);
        this.computerUsageRepository = new
ComputerUsageRepository(db);
        this.windowRepository = new WindowRepository(db);
        this.monitoringDaysRepository = new
MonitoringDaysRepository(db);
    }

    public ProcessorRepository getProcessorRepository() { ...

    public MemoryRepository getMemoryRepository() { ...

    public ComputerUsageRepository getComputerUsageRepository
() { ...

    public WindowRepository getWindowRepository() { ...

    public MonitoringDaysRepository
getMonitoringDaysRepository() { ...
}

```

Рисунок 2.9 - Клас MonitorRepositoryFactory

Шаблон “Visitor”

Патерн Visitor використано для форматування та представлення звітів у різних форматах. Він дозволяє додавати нові способи представлення даних без зміни логіки генерації звітів.

У системі реалізовано:

- ReportVisitor — базовий відвідувач;
- TextReportVisitor — формує звіт у вигляді тексту;
- JSONReportVisitor — формує звіт у JSON форматі.

Ці класи отримують дані від ReportService та повертають результат у відповідному форматі, забезпечуючи повне розділення логіки збору та логіки представлення.

```
public interface ReportVisitor {  
    Object visitDailyReport(Map<String, Object> data);  
    Object visitPeriodicReport(Map<String, Object> data);  
}
```

Рисунок 2.10 - Клас ReportVisitor

```

public class TextReportVisitor implements ReportVisitor {

    @SuppressWarnings("unchecked")
    private static <T> T cast(Object o) {
        return (T) o;
    }

    @Override
    public Object visitDailyReport(Map<String, Object> data) {

        if (data.containsKey(key: "error"))
            return "ERROR: " + data.get(key: "error");

        StringBuilder sb = new StringBuilder();
        sb.append(str: "==== DAILY REPORT ====\\n");
        sb.append(str: "Type: ").append(data.get
            (key: "type")).append(str: "\\n\\n");

        // 1) data → List<Map<String, Object>> (CPU / RAM /
        // Windows)
        if (data.containsKey(key: "data")) { ...

        // 2) programs → List<String>
        if (data.containsKey(key: "programs")) { ...

        // 3) uptime
        if (data.containsKey(key: "seconds")) { ...

        return sb.toString();
    }

    @Override
    public Object visitPeriodicReport(Map<String, Object>
data) {

        if (data.containsKey(key: "error"))
            return "ERROR: " + data.get(key: "error");
    }
}

```

Рисунок 2.11 - Клас TextReportVisitor


```

StringBuilder sb = new StringBuilder();
sb.append(str: "==== PERIODIC REPORT =====\n");
sb.append(str: "Type: ").append(data.get
(key: "type")).append(str: "\n");

if (data.containsKey(key: "period")) {
    Map<String, Object> period = cast(data.get
(key: "period"));

    sb.append(str: "Period: ")
      .append(period.get(key: "start"))
      .append(str: " → ")
      .append(period.get(key: "end"))
      .append(str: "\n\n");
} else {
    sb.append(str: "\n");
}

String type = (String) data.get(key: "type");

switch (type) {
    case "CPU Usage by Hours" -> ...
    case "Memory Usage by Hours" -> ...
    case "Programs Usage Time" -> ...
    case "Computer Uptime by Day(s)" -> ...
    case "Programs Used by Day(s)" -> ...
    default -> sb.append(str: "Raw: ").append(data);
}

return sb.toString();
}

private void formatCpuOrMemory(StringBuilder sb,
Map<String, Object> byDay) { ...

private void formatWindowTime(StringBuilder sb,
List<Map<String, Object>> list) { ...

```

Рисунок 2.12 - Клас TextReportVisitor

```

private void formatUptime(StringBuilder sb, Map<String,
Object> data) {

    if (data.containsKey(key: "days")) {
        Map<String, Object> days = cast(data.get
(key: "days"));

        for (var e : days.entrySet()) {
            sb.append(e.getKey())
                .append(str: " - ")
                .append(e.getValue())
                .append(str: " sec\n");
        }
        sb.append(str: "\n");
    }

    if (data.containsKey(key: "totalSeconds"))
        sb.append(str: "Total: ")
            .append(data.get(key: "totalSeconds"))
            .append(str: " sec (")
            .append(data.get(key: "totalFormatted"))
            .append(str: ")\n");
}

private void formatPrograms(StringBuilder sb, Map<String,
Object> byDay) {
    for (var entry : byDay.entrySet()) {
        sb.append(entry.getKey()).append(str: ":\n");

        List<String> list = cast(entry.getValue());

        for (String p : list)
            sb.append(str: " - ").append(p).append
(str: "\n");

        sb.append(str: "\n");
    }
}

```

Рисунок 2.13 - Клас TextReportVisitor

```

public class JSONReportVisitor implements ReportVisitor {

    private final Gson gson = new GsonBuilder()
        .setPrettyPrinting()
        .create();

    @Override
    public Object visitDailyReport(Map<String, Object> data) {
        return gson.toJson(data);
    }

    @Override
    public Object visitPeriodicReport(Map<String, Object>
data) {
        return gson.toJson(data);
    }
}

```

Рисунок 2.14 - Клас JSONReportVisitor

Шаблон “Bridge”

У даній реалізації шаблон Bridge не застосовується, оскільки структура системи не містить двох незалежних ієрархій абстракції та реалізації. Логіка створення моніторів повністю закрита шаблоном Abstract Factory, а поведінка моніторів визначається їхніми конкретними класами без окремої реалізаційної гілки.

Bridge був розглянутий на етапі проєктування, але не був використаний, оскільки його застосування не давало додаткових переваг для цієї системи і лише ускладнило б архітектуру.

2.3 Інструкція користувача

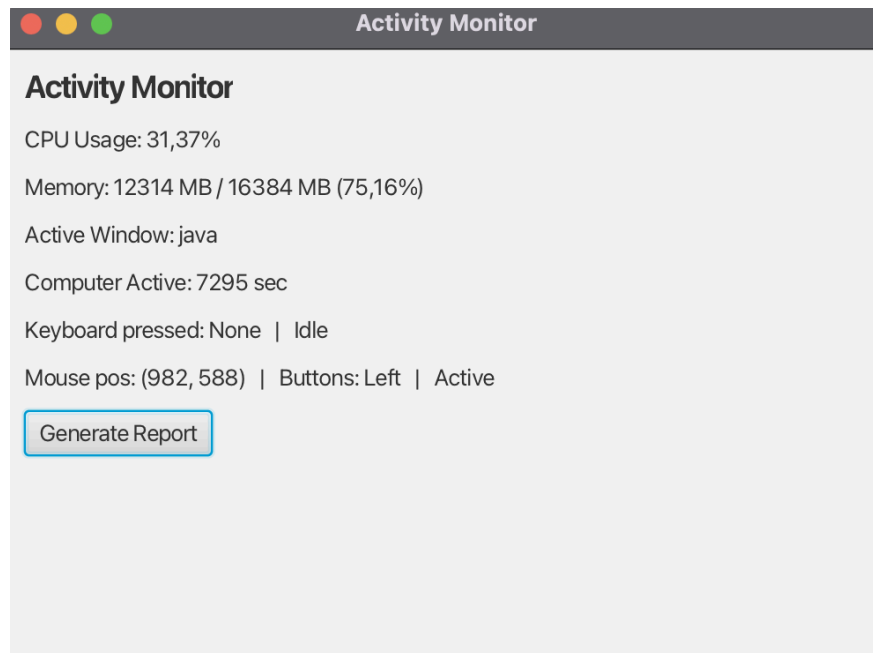


Рисунок 2.15 - Головна сторінка застосунку

Після натискання на кнопку “Generate Report” користувач бачить сторінку із налаштуванням звіту. Там можна обрати тип звіту, формат, дату/період дат, за який буде згенеровано звіт.

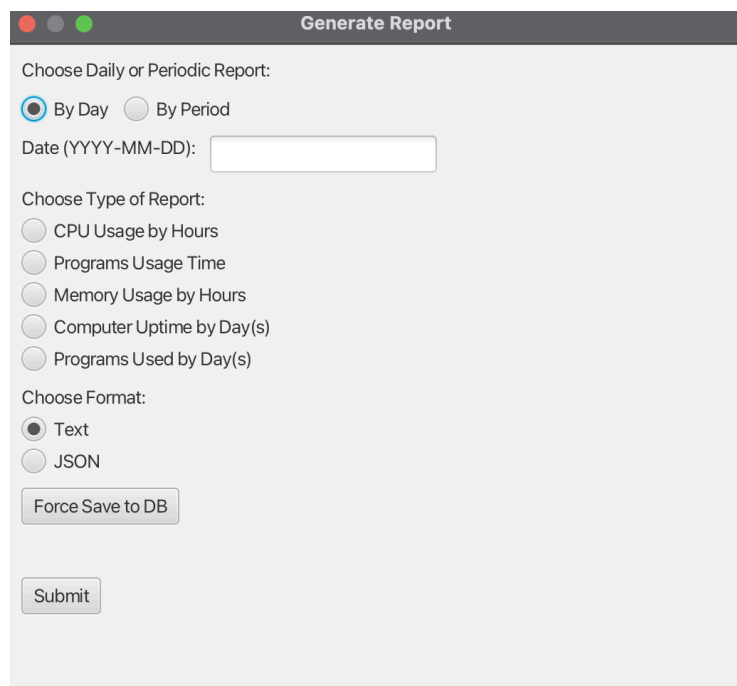


Рисунок 2.16 - Сторінка генерації репорту за певний день

Рисунок 2.17 - Сторінка генерації репорту за певний період дат

При натисканні кнопки “Submit” користувачу буде висвітлена сторінка що містить готовий звіт або інформацію про відсутність даних. Користувач може закрити цю сторінку при натисканні кнопки “Close”

Рисунок 2.18 - Згенерований репорт при відсутності даних

Рисунок 2.19 - Згенерований репорт при наявності даних

ВИСНОВКИ

В ході виконання курсової роботи було створено програмний застосунок для моніторингу активності комп'ютера, який забезпечує відстеження ключових системних показників: завантаження процесора, обсягу використаної оперативної пам'яті, активних вікон та тривалості роботи системи. Реалізований функціонал дає змогу автоматично збирати та накопичувати статистичні дані, що дозволяє користувачеві аналізувати стан комп'ютера та ефективність використання його ресурсів у різні моменти часу.

У процесі розробки було проаналізовано наявні засоби моніторингу, що дало можливість визначити оптимальні методи та підходи для створення власної системи. Результатом став програмний продукт з інтуїтивним інтерфейсом, який відображає поточні показники роботи та забезпечує формування звітів на основі зібраних даних. Така система може бути корисною як у побутовому, так і у робочому середовищі, де потрібний контроль за станом обладнання.

Підсумовуючи, поставлені завдання курсової роботи виконані в повному обсязі, що підтверджує важливість та практичну значущість обраної тематики. Розроблений монітор активності комп'ютера може слугувати інструментом для своєчасного виявлення відхилень у роботі системи та покращення надійності й продуктивності комп'ютерного обладнання.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Діаграма прецедентів — Wikipedia.
https://uk.wikipedia.org/wiki/Діаграма_прецедентів
2. Сценарій використання — Wikipedia.
https://uk.wikipedia.org/wiki/Сценарій_використання
3. Діаграма класів — Wikipedia.
https://uk.wikipedia.org/wiki/Діаграма_класів
4. Діаграма розгортання — Wikipedia.
https://uk.wikipedia.org/wiki/Діаграма_розгортання
5. Типи сучасних баз даних: короткий опис схем і приклади БД — Senior.ua.
<https://senior.ua/articles/11-tipv-suchasnih-baz-danih-korotkiy-opis-shemi--prikлади-bd>
6. Основні типи програмної архітектури — Art of BA.
<https://www.artofba.com/uk/post/main-types-of-software-architecture>
7. Abstract Factory — Refactoring Guru.
<https://refactoring.guru/uk/design-patterns/abstract-factory>
8. Visitor — Refactoring Guru.
<https://refactoring.guru/uk/design-patterns/visitor>
9. Iterator — Refactoring Guru.
<https://refactoring.guru/uk/design-patterns/iterator>
10. Command — Refactoring Guru.
<https://refactoring.guru/uk/design-patterns/command>
11. Richard Helm, Erich Gamma, Ralph Johnson, John Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional, 1994.
12. Роберт Мартін. Чистий код. Створення, аналіз та рефакторинг. Фабула #Pro, 2019.
13. Офіційна документація SQLite.
<https://www.sqlite.org/docs.html>

14.Офіційна документація JavaFX.

<https://openjfx.io/>

15.Офіційна документація Java Platform Standard Edition.

<https://docs.oracle.com/javase/>

16.Java Native Access (JNA) — офіційна документація.

<https://github.com/java-native-access/jna>

17.PlantUML Documentation — Офіційна документація.

<https://plantuml.com/>

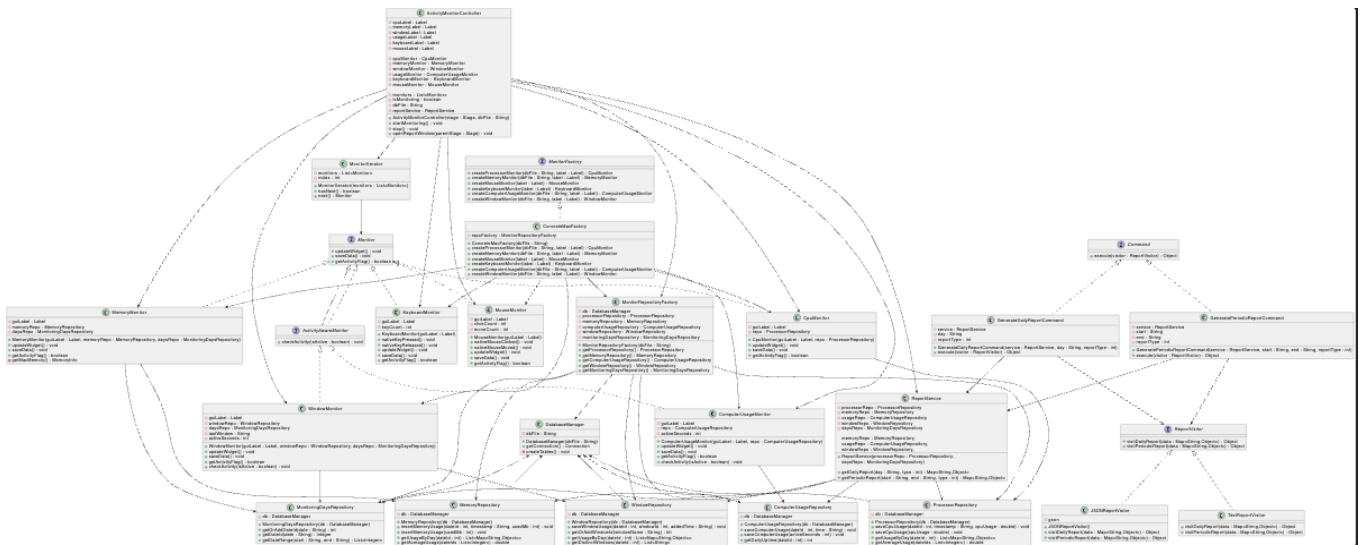
18.SQLiteStudio – офіційний сайт інструменту для роботи з SQLite.

<https://sqlitestudio.pl/>

ДОДАТКИ

Додаток А

Діаграма класів (повна)



Додаток Б

Вихідний код

Повний вихідний код знаходиться в репозиторії за посиланням :

<https://github.com/iamthegoose/system-activity-monitor>